

Hướng dẫn MVC qua module Quản lý ga

Phiên bản ngày 01/10/2026, ứng với commit `9748dcd` trên nhánh `feat/database-and-station-admin`.

Tài liệu dành cho thành viên nhóm TrainGo sắp làm các màn hình quản trị tiếp theo, như Quản lý tàu (T03) và Sơ đồ toa (T06) trong `docs/plans/active/chia-viec-mvp.md`. Đọc xong, bạn sẽ:

- hiểu MVC trong Spring Boot qua một module thật đang chạy;
- dùng được màn hình `/admin/stations`;
- đọc được từng file code của module và biết vì sao nó được viết như vậy;
- tự dựng một màn hình mới theo cùng mẫu.

Nội dung:

1. MVC là gì trong dự án này
2. Cách sử dụng màn hình Quản lý ga
3. Đi qua code từng tầng
4. Một request chạy như thế nào
5. Lỗi thường gặp
6. Làm màn hình mới theo mẫu này
7. Bài tập

Đây là tài liệu của phiên làm việc, không phải bản gốc. Khi có mâu thuẫn, code và `docs/product/`, `docs/decisions/` mới là bản đúng.

1. MVC là gì trong dự án này

Ý chính

MVC chia việc xử lý một trang web thành ba vai:

Vai	Trong Spring Boot của TrainGo	Ở module ga
Controller	Class có <code>@Controller</code> , mỗi method gắn với một URL	<code>StationAdminController</code>
Model	Dữ liệu controller đưa sang view qua đối tượng <code>Model</code>	<code>stations</code> , <code>stationForm</code> , <code>activeNav</code> , <code>successMessage</code>
View	Template Thymeleaf trong <code>src/main/resources/templates/</code>	<code>admin/stations.html</code> , <code>admin/station-form.html</code>

Để controller luôn mỏng (`AGENTS.md` yêu cầu), dự án tách thêm các tầng sau:

Tầng	Trách nhiệm	Ở module ga
Service	Nghiệp vụ và transaction	<code>StationService</code>
Repository	Đọc và ghi database	<code>StationRepository</code>
Entity	Một dòng trong bảng database	<code>Station</code>
Form	Dữ liệu người dùng gửi lên từ form	<code>StationForm</code>

Ba câu cần nhớ:

- Controller không chứa nghiệp vụ, chỉ nhận request, gọi service và chọn view.
- View không gọi database, chỉ hiển thị những gì có trong Model.
- Service không biết HTML, chỉ làm việc với dữ liệu và luật.

Đường đi của một request

Khi bạn mở `/admin/stations` :

```

Trình duyệt
  | GET /admin/stations
  v
StationAdminController.list()      Controller: nhận request
  | stationService.findAll()
  v
StationService.findAll()           Service: transaction chỉ đọc
  | stationRepository.findAllByNameAsc()
  v
StationRepository -> MySQL          Repository: SELECT bảng stations
  | List<Station>
  v
model.addAttribute("stations", ...) Model: dữ liệu cho view
  | return "admin/stations"
  v
templates/admin/stations.html      View: dựng HTML gửi trình duyệt

```

Các file của module

```

app/src/main/
├─ java/com/voyagego/traingo/
│   │   └─ station/
│   │       ├── controller/StationAdminController.java
│   │       ├── model/Station.java
│   │       ├── model/StationForm.java
│   │       ├── repository/StationRepository.java
│   │       └─ service/StationService.java, DuplicateStationCodeException,
│   │           StationInUseException, StationNotFoundException
│   └─ route/repository/RouteRepository.java (dùng để chặn xóa)
└─ admin/controller/AdminShellModelAdvice.java
└─ resources/
    ├── db/migration/V1__create_schema.sql      (bảng stations)
    ├── db/migration/V2__seed_demo_data.sql     (8 ga mẫu)
    ├── templates/admin/stations.html
    ├── templates/admin/station-form.html
    └─ static/js/admin/admin.js                 (hộp xác nhận xóa)

```

Mỗi feature có một package riêng (`station` , `train` , `route` ...). Bên trong chia tiếp `controller` , `model` , `repository` , `service` . Nhờ vậy hai người làm hai feature khác nhau hầu như không sửa chung file nào (`docs/decisions/0001-cau-truc-du-an-va-tech-stack.md`).

2. Cách sử dụng màn hình Quản lý ga

Chạy ứng dụng

Mở Docker Desktop trước, rồi trong PowerShell:

```
cd app
docker compose up -d          # MySQL; chạy lại sau mỗi lần khởi động máy
.\mvnw.cmd spring-boot:run    # lần đầu, Flyway tạo bảng và nạp seed
```

Mở `http://localhost:8081/admin/stations` và đăng nhập bằng `admin` / `admin`. Đây là tài khoản tạm khai trong `application.properties`, dùng cho tới khi làm xong Auth. Các tài khoản trong bảng `users` (như `admin@traingo.vn`) chưa đăng nhập được.

Sau khi đăng nhập, địa chỉ có thể kèm đuôi `?continue`. Đó là cách Spring Security 6 đưa bạn về trang đang mở dở, không phải lỗi.

Các thao tác

Thao tác	Cách làm	Kết quả
Xem	Mở <code>/admin/stations</code>	Bảng ga sắp theo tên, có ô tìm kiếm, sắp xếp cột, phân trang
Thêm	Nút "Thêm ga", điền form, bấm "Thêm ga"	Về danh sách, thông báo xanh "Đã thêm Ga Buôn Ma Thuột (BMT)."
Sửa	Biểu tượng bút chì ở cuối dòng	Form điền sẵn; lưu xong có thông báo "Đã cập nhật ..."
Xóa	Biểu tượng thùng rác, chọn OK ở hộp xác nhận	"Đã xóa ...", hoặc thông báo đỏ nếu ga đang được dùng

Quy tắc dữ liệu

Trường	Bắt buộc	Quy tắc
Mã ga	Có	Chữ cái không dấu và chữ số, tối đa 10 ký tự, tự viết hoa, không trùng
Tên ga	Có	Tối đa 100 ký tự
Tỉnh / Thành phố	Không	Tối đa 100 ký tự
Địa chỉ	Không	Tối đa 255 ký tự

Khoảng trắng ở đầu và cuối mỗi ô được tự bỏ. Ô để trống được lưu là không có giá trị (`NULL`). Vì mã luôn được viết hoa, gõ `sgn` khi đã có `SGN` sẽ bị báo trùng.

Ga đang là điểm đi hoặc điểm đến của một tuyến thì không xóa được. Trong dữ liệu mẫu, chỉ Quy Nhơn (`QNH`), Huế (`HUE`) và Vinh (`VIN`) xóa được; năm ga còn lại đều có tuyến.

Xem dữ liệu trong database

Máy dev không có MySQL client, nên dùng client có sẵn trong container:

```
docker exec -it traingo-mysql mysql -utraingo -ptraingo `
--default-character-set=utf8mb4 traingo
```

Rồi gõ `SELECT * FROM stations;`. Thiếu `--default-character-set=utf8mb4` thì chữ tiếng Việt hiện sai.

3. Đi qua code từng tầng

Các đoạn code dưới đây chép từ code thật. Một số dòng được ngắt lại cho vừa khổ giấy, và phần lặp lại như getter được rút gọn bằng chú thích.

3.1 Bảng `stations` trong migration

```
CREATE TABLE stations (  
  id      BIGINT      NOT NULL AUTO_INCREMENT,  
  code    VARCHAR(10) NOT NULL,  
  name    VARCHAR(100) NOT NULL,  
  city    VARCHAR(100),  
  address VARCHAR(255),  
  CONSTRAINT pk_stations PRIMARY KEY (id),  
  CONSTRAINT uk_stations_code UNIQUE (code)  
);
```

- Bảng nằm trong `V1__create_schema.sql`. Flyway chạy file này khi ứng dụng khởi động lần đầu trên database trống (`docs/decisions/0005-quan-ly-schema-bang-flyway.md`).
- `uk_stations_code` là lớp chặn cuối cùng: kể cả khi code quên kiểm tra, MySQL vẫn từ chối mã trùng.
- Bảng `routes` có khóa ngoại trỏ về `stations` và không có `ON DELETE`. Nên MySQL cũng từ chối xóa một ga đang có tuyến đường.
- Muốn đổi bảng thì viết migration mới (`V3__...`), không sửa file đã chạy.

3.2 Entity Station

```
@Entity
@Table(name = "stations", uniqueConstraints = @UniqueConstraint(
    name = "uk_stations_code", columnNames = "code"))
public class Station {

    public static final int CODE_MAX_LENGTH = 10;
    public static final int NAME_MAX_LENGTH = 100;
    public static final int CITY_MAX_LENGTH = 100;
    public static final int ADDRESS_MAX_LENGTH = 255;

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false, length = CODE_MAX_LENGTH)
    private String code;
    @Column(nullable = false, length = NAME_MAX_LENGTH)
    private String name;
    @Column(length = CITY_MAX_LENGTH)
    private String city;
    @Column(length = ADDRESS_MAX_LENGTH)
    private String address;

    protected Station() {
    }

    public Station(String code, String name, String city,
        String address) {
        update(code, name, city, address);
    }

    public void update(String code, String name, String city,
        String address) {
        this.code = normalizeCode(code);
        this.name = name;
        this.city = city;
        this.address = address;
    }

    public static String normalizeCode(String code) {
        return code.trim().toUpperCase(Locale.ROOT);
    }

    // getId(), getCode(), getName(), getCity(), getAddress()
}
```

- `@Entity` và `@Table` nối class này với bảng `stations`. Mỗi object `Station` là một dòng trong bảng.

- Tên field theo kiểu `camelCase` ; Spring Boot tự đổi thành tên cột `snake_case` khi cần. Bốn field ở đây trùng tên cột nên không cần đổi gì.
- `@Id` và `@GeneratedValue(strategy = GenerationType.IDENTITY)` : id do MySQL tự sinh (`AUTO_INCREMENT`), code không bao giờ tự đặt id.
- Class không có setter. Muốn đổi dữ liệu thì phải đi qua `update(...)` , và `update` luôn gọi `normalizeCode` . Nhờ vậy luật "mã luôn viết hoa" chỉ nằm ở một chỗ và không thể bị bỏ qua.
- `Locale.ROOT` giúp việc viết hoa không phụ thuộc ngôn ngữ cài trên máy chạy.
- `protected Station()` là constructor rỗng mà Hibernate cần khi đọc một dòng từ database ra object. Để `protected` thì code khác không tạo được ga rỗng.
- Bốn hằng số độ dài được `StationForm` dùng lại, nên giới hạn ở form luôn khớp với độ dài cột.
- `spring.jpa.hibernate.ddl-auto=validate` : lúc khởi động, Hibernate kiểm tra class này khớp bảng. Lệnh là ứng dụng dừng ngay, không đợi tới lúc một trang bị lỗi.

3.3 Form `StationForm`

```
public class StationForm {

    @NotBlank(message = "Vui lòng nhập mã ga.")
    @Size(max = Station.CODE_MAX_LENGTH,
        message = "Mã ga tối đa {max} ký tự.")
    @Pattern(regexp = "[A-Za-z0-9]+",
        message = "Mã ga chỉ gồm chữ cái không dấu và chữ số.")
    private String code;

    @NotBlank(message = "Vui lòng nhập tên ga.")
    @Size(max = Station.NAME_MAX_LENGTH,
        message = "Tên ga tối đa {max} ký tự.")
    private String name;

    @Size(max = Station.CITY_MAX_LENGTH,
        message = "Tỉnh / thành phố tối đa {max} ký tự.")
    private String city;

    @Size(max = Station.ADDRESS_MAX_LENGTH,
        message = "Địa chỉ tối đa {max} ký tự.")
    private String address;

    public static StationForm from(Station station) {
        StationForm form = new StationForm();
        form.code = station.getCode();
        form.name = station.getName();
        form.city = station.getCity();
        form.address = station.getAddress();
        return form;
    }

    // getter và setter cho cả bốn field
}
```

- Form tách khỏi entity vì lý do an toàn: request chỉ đặt được đúng bốn field này. Nếu bind thẳng vào `Station`, kẻ xấu có thể gửi thêm `id` để ghi đè lên một ga khác.
- Annotation validation đến từ Jakarta Validation. `{max}` trong thông báo được thay bằng số thật, ví dụ "Mã ga tối đa 10 ký tự."
- `@Pattern` khớp với toàn bộ chuỗi, nên `SG N` (có khoảng trắng) hay `Sài` (có dấu) đều không hợp lệ.
- Giá trị `null` được `@Size` và `@Pattern` coi là hợp lệ. Bắt buộc nhập là việc của `@NotBlank`, nên mã trống chỉ hiện một thông báo "Vui lòng nhập mã ga."
- Setter là bắt buộc: Spring gọi `setCode(...)`, `setName(...)` để đổ dữ liệu từ request vào form.
- `from(...)` dùng khi mở form sửa, để điền sẵn dữ liệu của ga đang sửa.

3.4 Repository `StationRepository`

```
public interface StationRepository
    extends JpaRepository<Station, Long> {

    List<Station> findAllByOrderByNameAsc();

    boolean existsByCode(String code);

    boolean existsByCodeAndIdNot(String code, Long id);
}
```

Không có dòng SQL nào: Spring Data đọc tên method rồi tự sinh câu truy vấn.

Method	Câu truy vấn tương ứng	Dùng khi
<code>findAllByOrderByNameAsc()</code>	<code>SELECT ... ORDER BY name ASC</code>	Hiển thị danh sách
<code>existsByCode(code)</code>	<code>SELECT ... WHERE code = ?</code>	Thêm ga: mã đã có chưa
<code>existsByCodeAndIdNot(code, id)</code>	<code>... WHERE code = ? AND id <> ?</code>	Sửa ga: mã có thuộc ga khác không

`existsByCodeAndIdNot` bỏ qua chính ga đang sửa. Nếu dùng `existsByCode` khi sửa, việc lưu lại ga mà giữ nguyên mã sẽ bị báo trùng với chính nó.

`JpaRepository` cho sẵn `findById`, `save`, `delete`, `flush` ... nên không phải viết lại.

Ở package `route`, `RouteRepository` có thêm một method mà module ga dùng:

```
boolean existsByOriginStationIdOrDestinationStationId(
    Long originStationId, Long destinationStationId);
```

`OriginStationId` được Spring Data hiểu là `originStation.id`, tức cột `origin_station_id`. Method trả `true` nếu có tuyến bắt đầu hoặc kết thúc ở ga đó.

3.5 Service `StationService`

```
@Service
public class StationService {

    private final StationRepository stationRepository;
    private final RouteRepository routeRepository;

    public StationService(StationRepository stationRepository,
        RouteRepository routeRepository) {
        this.stationRepository = stationRepository;
        this.routeRepository = routeRepository;
    }

    @Transactional(readOnly = true)
    public List<Station> findAll() {
        return stationRepository.findAllByOrderByNameAsc();
    }

    @Transactional(readOnly = true)
    public Station findById(long id) {
        return stationRepository.findById(id)
            .orElseThrow(() -> new StationNotFoundException(id));
    }

    @Transactional
    public Station create(StationForm form) {
        String code = Station.normalizeCode(form.getCode());
        if (stationRepository.existsByCode(code)) {
            throw new DuplicateStationCodeException(code);
        }
        return stationRepository.save(new Station(code,
            form.getName(), form.getCity(), form.getAddress()));
    }
}
```

- `@Service` cho Spring biết đây là một bean để tiêm (inject) vào controller.
- Các dependency được nhận qua constructor, không dùng `@Autowired` trên field (quy tắc trong `AGENTS.md`). Field `final` nên không thể vô tình bị đổi.
- `@Transactional` gom mọi thao tác database trong method thành một transaction: cùng thành công hoặc cùng bị hủy. `readOnly = true` báo cho Hibernate biết method chỉ đọc, để bỏ qua bước theo dõi thay đổi.
- `findById` ném `StationNotFoundException` khi không có ga. Exception này mang `@ResponseStatus(HttpStatus.NOT_FOUND)`, nên người dùng nhận trang 404.
- `create` kiểm tra mã trùng trước khi lưu, để báo lỗi dễ hiểu. Nếu hai người bấm lưu cùng một lúc thì cả hai đều qua được bước kiểm tra; khi đó khóa unique `uk_stations_code` chặn người thứ hai.

```

@Transactional
public Station update(long id, StationForm form) {
    Station station = findById(id);
    String code = Station.normalizeCode(form.getCode());
    if (stationRepository.existsByCodeAndIdNot(code, id)) {
        throw new DuplicateStationCodeException(code);
    }
    station.update(code, form.getName(), form.getCity(),
        form.getAddress());
    stationRepository.flush();
    return station;
}

@Transactional
public Station delete(long id) {
    Station station = findById(id);
    if (routeRepository
        .existsByOriginStationIdOrDestinationStationId(id, id)) {
        throw new StationInUseException(station);
    }
    stationRepository.delete(station);
    return station;
}
}

```

- `update` không gọi `save()` mà dữ liệu vẫn được lưu. Đây là cơ chế dirty checking của Hibernate: object `station` lấy ra trong transaction được Hibernate theo dõi. Đổi field rồi kết thúc method là Hibernate tự sinh câu `UPDATE`.
- `flush()` ép câu `UPDATE` chạy ngay trong method. Nếu mã vừa bị ai đó chiếm, lỗi xuất hiện tại đây với cùng loại exception mà `create` gặp, thay vì nổ ra lúc commit dưới một dạng khác.
- `delete` kiểm tra tuyến trước rồi mới xóa. Cách "cứ xóa rồi bắt lỗi khóa ngoại" không dùng được: exception từ repository đánh dấu transaction là phải rollback, nên dù có bắt được exception, lúc commit Spring vẫn ném `UnexpectedRollbackException`.
- Ba exception của module đều nằm cạnh service: `DuplicateStationCodeException` (mã trùng), `StationInUseException` (ga đang được dùng, mang theo ga để controller in tên) và `StationNotFoundException` (404).

3.6 Controller `StationAdminController`

```
@Controller
@RequestMapping("/admin/stations")
public class StationAdminController {

    private static final String LIST_VIEW = "admin/stations";
    private static final String FORM_VIEW = "admin/station-form";
    private static final String REDIRECT_TO_LIST =
        "redirect:/admin/stations";

    private final StationService stationService;

    public StationAdminController(StationService stationService) {
        this.stationService = stationService;
    }

    @InitBinder
    void trimSubmittedText(WebDataBinder binder) {
        binder.registerCustomEditor(String.class,
            new StringTrimmerEditor(true));
    }

    @ModelAttribute("activeNav")
    String activeNav() {
        return "stations";
    }

    @GetMapping
    public String list(Model model) {
        model.addAttribute("stations", stationService.findAll());
        return LIST_VIEW;
    }

    @GetMapping("/new")
    public String createForm(Model model) {
        model.addAttribute("stationForm", new StationForm());
        return FORM_VIEW;
    }
}
```

- `@Controller` trả về tên view, rồi Thymeleaf dựng HTML từ view đó. Khác với `@RestController`, loại trả thẳng JSON (T06 sẽ dùng loại này cho API).
- `@RequestMapping("/admin/stations")` là tiền tố chung, nên `@GetMapping("/new")` ứng với `/admin/stations/new`.
- Chuỗi trả về như `"admin/stations"` là đường dẫn template, tính từ `templates/` và bỏ đuôi `.html`.

- `@InitBinder` chạy trước khi dữ liệu form được đổ vào `StationForm`. `StringTrimmerEditor(true)` cắt khoảng trắng hai đầu và đổi chuỗi rỗng thành `null`. Vì vậy `"SGN "` được kiểm tra như `"SGN"`, và ô Tỉnh để trống được lưu thành `NULL`.
- Method gắn `@ModelAttribute("activeNav")` chạy trước mọi handler trong class và đặt `activeNav = "stations"` vào Model. Sidebar đọc giá trị này để làm sáng mục "Quản lý ga".

```
@PostMapping
public String create(
    @Valid @ModelAttribute("stationForm") StationForm form,
    BindingResult bindingResult,
    RedirectAttributes redirectAttributes) {
    if (bindingResult.hasErrors()) {
        return FORM_VIEW;
    }
    Station station;
    try {
        station = stationService.create(form);
    } catch (DuplicateStationCodeException
        | DataIntegrityViolationException ex) {
        rejectDuplicateCode(bindingResult);
        return FORM_VIEW;
    }
    redirectAttributes.addFlashAttribute("successMessage",
        "Đã thêm " + describe(station) + ".");
    return REDIRECT_TO_LIST;
}
```

- `@ModelAttribute("stationForm")` gom các field của request thành một `StationForm`, đồng thời đặt nó vào Model với tên `stationForm`. Nhờ vậy khi form hiện lại, những gì người dùng đã gõ vẫn còn.
- `@Valid` chạy các luật trong `StationForm`. Kết quả nằm trong `BindingResult`. Tham số `BindingResult` phải đứng ngay sau tham số có `@Valid`; đặt sai chỗ thì Spring trả lỗi 400 thay vì cho bạn hiện lại form.
- Có lỗi thì `return FORM_VIEW`: hiện lại form, không redirect, và service không hề được gọi.
- Mã trùng không phải lỗi về hình dạng dữ liệu, nên chỉ service mới phát hiện được. Controller bắt exception rồi gắn lỗi vào đúng ô "Mã ga" bằng `bindingResult.rejectValue("code", ...)`.
- Bắt `DataIntegrityViolationException` ở đây là an toàn, vì transaction của service đã kết thúc và đã rollback xong.
- Thành công thì `addFlashAttribute` lưu thông báo, rồi `redirect`: khiến trình duyệt gửi một request `GET` mới. Mẫu này gọi là Post/Redirect/Get: bấm F5 ở danh sách chỉ tải lại trang, không thêm ga lần nữa. Flash attribute chỉ sống qua đúng một lần redirect.

```
@GetMapping("/{id}/edit")
public String editForm(@PathVariable long id, Model model) {
    model.addAttribute("stationId", id);
    model.addAttribute("stationForm",
        StationForm.from(stationService.findById(id)));
    return FORM_VIEW;
}

@PostMapping("/{id}/delete")
public String delete(@PathVariable long id,
    RedirectAttributes redirectAttributes) {
    try {
        Station station = stationService.delete(id);
        redirectAttributes.addFlashAttribute("successMessage",
            "Đã xóa " + describe(station) + ".");
    } catch (StationInUseException ex) {
        redirectAttributes.addFlashAttribute("errorMessage",
            "Không xóa được " + describe(ex.getStation())
            + " vì ga đang là điểm đi hoặc điểm đến"
            + " của một tuyến.");
    }
    return REDIRECT_TO_LIST;
}
```

- `@PathVariable long id` lấy số trong URL, ví dụ `7` trong `/admin/stations/7/edit`.
- Hai thao tác thêm và sửa dùng chung `station-form.html`. Chỉ khi sửa mới có `stationId` trong Model, và template dựa vào đó để biết form gửi về đâu.
- `update(...)` (không chép ở đây) giống `create(...)`, chỉ khác là đặt `stationId` vào Model trước. Nhờ vậy khi form có lỗi, nó vẫn gửi về đúng URL của ga đang sửa.
- Xóa dùng `POST` chứ không dùng link `GET`, vì thao tác làm thay đổi dữ liệu không được nằm sau một đường link mà trình duyệt hay công cụ có thể tự mở.

Tổng hợp các route:

Method	URL	Handler	Kết quả
GET	/admin/stations	list	Trang danh sách
GET	/admin/stations/new	createForm	Form trống
POST	/admin/stations	create	Redirect, hoặc form kèm lỗi
GET	/admin/stations/{id}/edit	editForm	Form điền sẵn, hoặc 404
POST	/admin/stations/{id}	update	Redirect, hoặc form kèm lỗi
POST	/admin/stations/{id}/delete	delete	Redirect kèm thông báo xanh hoặc đỏ

3.7 View stations.html

```
<html lang="vi" xmlns:th="http://www.thymeleaf.org"
  th:replace="~{admin/layout/base :: page(~{::title}, ~{::main})}">
...
<div th:if="${successMessage}" class="alert alert-success ...">
  <span th:text="${successMessage}">Đã thêm Ga Sài Gòn (SGN).</span>
</div>
...
<tr th:each="station : ${stations}">
  <td class="fw-semibold" th:text="${station.code}">SGN</td>
  <td th:text="${station.name}">Ga Sài Gòn</td>
  ...
  <a class="btn btn-light btn-sm" ...
    th:href="@{/admin/stations/{id}/edit(id=${station.id})}">
    <i class="bi bi-pencil"></i>
  </a>
  <form method="post" class="d-inline"
    th:action="@{/admin/stations/{id}/delete(id=${station.id})}"
    th:data-confirm="|Xóa ${station.name} (${station.code})? ...|">
    <button type="submit" class="btn btn-light btn-sm" ...>
      <i class="bi bi-trash"></i>
    </button>
  </form>
</tr>
```

- `th:replace` lồng `<title>` và `<main>` của trang vào khung admin chung (`admin/layout/base.html`), nên trang không phải tự khai header và sidebar.
- `${...}` đọc dữ liệu trong Model. `th:if` chỉ hiện khối khi giá trị khác `null`; `th:each` lặp qua danh sách.

- Chữ mẫu như `SGN` hay `Ga Sài Gòn` trong template bị thay khi render. Nó chỉ để mở file HTML trực tiếp vẫn đọc được.
- `@{...}` dựng URL. `{id}` được thay bằng `station.id`, nên ra `/admin/stations/7/edit`.
- Form xóa phải dùng `th:action`, không dùng `action`. Spring Security bật bảo vệ CSRF: mọi `POST` phải mang theo một token bí mật. `th:action` tự chèn token đó vào form; thiếu token thì server trả 403.
- `|...|` là chuỗi có biến bên trong. `th:data-confirm` tạo attribute `data-confirm`, và `admin.js` đọc nó để hỏi lại trước khi xóa. Câu hỏi đầy đủ là "Xóa Ga Sài Gòn (SGN)? Thao tác này không hoàn tác được."
- Dấu `...` trong đoạn trích là chỗ được lược bớt, ví dụ các attribute `th:title`, `th:aria-label` giúp trình đọc màn hình đọc được nút.
- Bảng có class `datatable`, nên `admin.js` gắn thêm tìm kiếm, sắp xếp và phân trang mà trang không phải viết thêm dòng nào.

3.8 View `station-form.html`

```
<main th:with="editing=${stationId != null}">
  ...
  <form method="post" novalidate th:object="${stationForm}"
    th:action="${editing}
      ? @{/admin/stations/{id}(id=${stationId})}
      : @{/admin/stations}">
    <label for="code" class="form-label">
      Mã ga <span class="text-danger">*</span></label>
    <input type="text" class="form-control" th:field="*{code}"
      th:errorclass="is-invalid" maxlength="10" autocomplete="off"
      required aria-describedby="codeHelp">
    <div class="invalid-feedback" th:errors="*{code}">
      Vui lòng nhập mã ga.</div>
    ...
    <button type="submit" class="btn btn-primary">
      <i class="bi bi-check-lg me-1"></i><span
        th:text="${editing} ? 'Lưu thay đổi' : 'Thêm ga'">Thêm ga</span>
    </button>
  </form>
</main>
```

- `th:with` tạo biến `editing` dùng trong cả trang: `true` khi có `stationId`, tức là đang sửa.
- `th:object="${stationForm}"` chọn object gốc. Bên trong, `*{code}` nghĩa là `stationForm.code`.
- `th:field="*{code}"` đặt luôn ba attribute `id="code"`, `name="code"` và `value` bằng giá trị hiện tại. Khi form hiện lại vì lỗi, `value` chính là những gì người dùng vừa gõ.
- `th:errorclass="is-invalid"` chỉ thêm class khi field có lỗi. `th:errors` in thông báo lỗi, và Bootstrap chỉ hiện `invalid-feedback` khi ô đứng trước nó có class `is-invalid`.

- `novalidate` tắt kiểm tra sẵn có của trình duyệt. Mọi luật được server kiểm tra và báo bằng tiếng Việt ngay dưới ô, thay vì bong bóng theo ngôn ngữ trình duyệt. `maxlength` vẫn chặn người dùng gõ quá dài.

3.9 Hộp xác nhận trong `admin.js`

```
document.addEventListener('submit', function (event) {  
    const message = event.target.dataset  
        ? event.target.dataset.confirm : undefined;  
    if (message && !window.confirm(message)) {  
        event.preventDefault();  
    }  
});
```

- Lắng nghe sự kiện `submit` trên `document` chứ không gắn vào từng form (kỹ thuật event delegation). Datableable vẽ lại các dòng mỗi khi sắp xếp hay chuyển trang, nên listener gắn trên từng dòng sẽ mất.
- `dataset.confirm` đọc attribute `data-confirm`. Form nào có attribute này đều được hỏi lại; bấm Hủy thì `preventDefault()` chặn việc gửi form.
- Màn hình mới chỉ cần thêm `th:data-confirm` vào form là có hộp xác nhận, không phải viết JavaScript.

3.10 `AdminShellModelAdvice`

```
@ControllerAdvice  
public class AdminShellModelAdvice {  
  
    @ModelAttribute("adminName")  
    String adminName() {  
        return "Quản trị viên";  
    }  
}
```

- Header admin hiển thị `${adminName}` trên mọi trang. `@ControllerAdvice` kèm `@ModelAttribute` đưa giá trị này vào Model của mọi controller. Vì vậy controller ở package `station` hay `train` không phải tự khai.
- Đây là giá trị tạm. Khi làm Auth, header sẽ đọc tên người đang đăng nhập và class này bị xóa.

4. Một request chạy như thế nào

4.1 Thêm ga thành công

Ví dụ người dùng gõ mã `bmt` kèm khoảng trắng ở hai đầu, tên `Ga Buôn Ma Thuột`, tỉnh `Đắk Lắk`, và để trống địa chỉ.

Trình duyệt		Server
GET /admin/stations/new	----->	createForm(): StationForm rỗng
	<-----	200, HTML form
POST /admin/stations	----->	create(): kiểm tra, INSERT, flash
(code, name, city, address, _csrf)		
	<-----	302, Location: /admin/stations
GET /admin/stations	----->	list(): SELECT
	<-----	200, HTML + "Đã thêm ..."

1. Spring Security kiểm tra phiên đăng nhập và token `_csrf` của `POST`.
2. `@InitBinder` cắt khoảng trắng: mã thành `bmt`, địa chỉ rỗng thành `null`.
3. `@Valid` kiểm tra `StationForm`: mọi luật đều đạt.
4. `StationService.create` chuẩn hóa mã thành `BMT`, `existsByCode("BMT")` trả `false`, rồi `save` chạy câu `INSERT`.
5. Transaction commit, dòng mới nằm trong MySQL.
6. Controller đặt flash `Đã thêm Ga Buôn Ma Thuột (BMT)`. và trả `redirect:`, trình duyệt nhận mã 302.
7. Trình duyệt tự gửi `GET /admin/stations`. Trang danh sách hiện ga mới cùng thông báo xanh. Bấm F5 lúc này chỉ gửi lại `GET`, không tạo ga thứ hai.

4.2 Form không hợp lệ

Người dùng bấm "Thêm ga" mà để trống mã và tên.

1. `@Valid` phát hiện hai lỗi `@NotBlank` và ghi chúng vào `BindingResult`.
2. `bindingResult.hasErrors()` là `true`, nên controller trả `FORM_VIEW` với status 200. Không có redirect, và service không được gọi.
3. Template thấy lỗi: thêm viền đỏ (`is-invalid`) và in "Vui lòng nhập mã ga.", "Vui lòng nhập tên ga." dưới hai ô.

4.3 Mã trùng

Người dùng gõ mã `sgn` trong khi `SGN` đã có.

1. Luật trong form đều đặt: `sgn` là chữ cái không dấu.
2. Service chuẩn hóa thành `SGN`, `existsByCode` trả `true`, nên ném `DuplicateStationCodeException`. Transaction rollback, không có gì được ghi.
3. Controller bắt exception, gọi `rejectValue("code", ...)` và hiện lại form với thông báo "Mã ga này đã được dùng cho một ga khác." dưới ô Mã ga.

4.4 Xóa ga đang có tuyến dùng

1. Người dùng bấm thùng rác ở dòng `SGN`. `admin.js` hỏi lại, người dùng chọn OK.
2. Form gửi `POST /admin/stations/{id}/delete` kèm token CSRF.
3. Service tìm thấy ga, `routeRepository` báo có tuyến dùng, nên ném `StationInUseException`.
Lệnh `DELETE` không bao giờ chạy.
4. Controller bắt exception, đặt flash `errorMessage` và redirect. Danh sách hiện thông báo đỏ "Không xóa được Ga Sài Gòn (SGN) vì ...", ga vẫn còn nguyên.

4.5 Ga không tồn tại

Mở `/admin/stations/999999/edit`: `findById` ném `StationNotFoundException`. Exception này có `@ResponseStatus(HttpStatus.NOT_FOUND)`, nên Spring trả trang lỗi 404 thay vì lỗi 500.

5. Lỗi thường gặp

Mỗi mục gồm cách viết sai, hậu quả, và cách đúng.

1. **Bind thẳng entity**, như `@ModelAttribute Station station`. Hậu quả: request gửi thêm `id` là ghi đè được một ga khác. Cách đúng: dùng form object riêng như `StationForm`.
2. **Trả view sau khi POST thành công**. Hậu quả: bấm F5 là gửi lại form, dữ liệu bị thêm hai lần. Cách đúng: `redirect:` kèm flash attribute.
3. **BindingResult không đứng ngay sau tham số @Valid**. Hậu quả: Spring trả lỗi 400 thay vì hiện lại form. Cách đúng: đặt `BindingResult` ngay sau tham số được validate.
4. **Bắt DataIntegrityViolationException bên trong method @Transactional**. Hậu quả: lúc commit, Spring ném `UnexpectedRollbackException`. Cách đúng: kiểm tra trước trong service, hoặc bắt exception ở controller.
5. **Form POST dùng action= thay vì th:action=**. Hậu quả: form thiếu token CSRF, server trả 403. Cách đúng: luôn dùng `th:action`.
6. **Quên method activeNav()**. Hậu quả: sidebar không sáng mục nào. Cách đúng: khai `@ModelAttribute("activeNav")` trong controller như module ga.
7. **Đọc relation LAZY trong template**, như `route.originStation.name`. Hậu quả: `LazyInitializationException`, vì dự án tắt `open-in-view`. Cách đúng: nạp đủ dữ liệu view cần ngay trong service rồi mới trả về.
8. **Sửa V1__create_schema.sql sau khi đã chạy**. Hậu quả: Flyway từ chối khởi động trên máy người khác. Cách đúng: viết migration mới `V3__...`.
9. **Nạp app.css trong trang admin**. Hậu quả: giao diện admin vỡ mà log không báo gì. Cách đúng: trang admin chỉ dùng khung `admin/layout/base.html`.
10. **Sửa template hay CSS mà trình duyệt chưa thấy thay đổi**. Ứng dụng đọc file từ `app/target/classes`, không đọc trực tiếp từ `src`. Cách làm: build lại project trong IDE, hoặc chạy `.\mvnw.cmd resources:resources` trong thư mục `app`, rồi tải lại trang.

6. Làm màn hình mới theo mẫu này

Ví dụ T03, Quản lý tàu. Entity `Train` và `TrainRepository` đã có sẵn, nên chỉ còn controller, form, service và view:

1. `train/model/TrainForm.java`: chép `StationForm`, giữ `code` và `name`, đổi thông báo cho hợp với tàu.
2. `train/service/TrainService.java`: `findAll`, `findById` (ném exception 404), `create`, `update` có kiểm tra mã trùng. Không có `delete`, vì tàu không có chức năng xóa. Nếu cần query mới thì thêm vào `TrainRepository`.
3. `train/controller/TrainAdminController.java` với `@RequestMapping("/admin/trains")`, `@InitBinder`, `activeNav()` trả `"trains"`, và các handler `list`, `createForm`, `create`, `editForm`, `update`.
4. `templates/admin/train-form.html`: chép `station-form.html` rồi đổi tên field và nhãn.
5. `templates/admin/trains.html`: bỏ `disabled` ở các nút, cho `th:each` đọc `Train` thật.
6. Gỡ phần dữ liệu giả của tàu khỏi `AdminController` và `AdminService`, rồi xóa `admin/model/Train.java`.
7. Chạy ứng dụng và thử lại năm tình huống của mục 4 trên màn hình tàu.
8. Thêm một dòng vào `docs/WORKLOG.md`, nằm trong cùng commit với code.

Không sửa entity, migration, `SecurityConfig` hay `admin/layout/base.html`. Cần đổi những file đó thì báo nhóm trước.

7. Bài tập

1. **Hai cách gõ cùng bị báo trùng.** Thêm một ga với mã `sgn` (chữ thường), rồi với mã `SGN` (có khoảng trắng). Hãy giải thích vì sao cả hai đều bị báo trùng, và mỗi trường hợp bị chặn ở dòng code nào. Gợi ý: một trường hợp đi qua `@InitBinder`, trường hợp kia qua `normalizeCode`.
2. **Dirty checking.** `StationService.update` không gọi `save()`. Hãy giải thích vì sao dữ liệu vẫn được lưu, và chuyện gì xảy ra nếu bỏ `@Transactional` khỏi method đó.
3. **Thêm cột "Số tuyến".** Thêm vào danh sách ga một cột cho biết ga là điểm đi hoặc điểm đến của bao nhiêu tuyến. Với seed hiện tại, kết quả đúng là: `SGN`, `DNG`, `HNI` có 4 tuyến; `NTR`, `LCI` có 2; `QNH`, `HUE`, `VIN` có 0. Gợi ý: dùng một query `@Query` gom nhóm trong `RouteRepository` để lấy số tuyến của mọi ga trong một lần, không query riêng từng ga.
4. **Viết unit test.** Viết test bằng Mockito cho `StationService.delete`: khi `routeRepository` trả `true` thì method ném `StationInUseException` và không gọi `stationRepository.delete`. Gợi ý: `verify(stationRepository, never()).delete(any())`. Class test phải có tên kết thúc bằng `Test`, nếu không Surefire sẽ bỏ qua mà không báo lỗi.